

Plan de Casos de Prueba - Sprint 2

01 Epica: Dashboard

#1 Obtener resumen de dinero disponible - GET /accounts/{id} (account-service) - Suite automatizada: AccountApiTest

ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC01	Obtener saldo correctamente	Usuario autenticado con token válido	Enviar GET /accounts/{id} con token válido	200 OK + saldo disponible	Pass
TC02	Obtener saldo sin token	API disponible	Enviar GET /accounts/{id} sin token	401 Unauthorized	Pass
TC03	Obtener saldo con token inválido	API disponible	Enviar GET con token inválido	401 Unauthorized	Pass
TC04	Obtener saldo con ID inexistente	Usuario autenticado	Enviar GET con ID inexistente	404 Not found - Account not found	Pass
TC05	Obtener saldo con ID inválido (string)	API disponible	Enviar GET /accounts/abc	400 Bad Request	Pass
TC06	Usuario accede a cuenta de otro usuario	Usuario autenticado	GET con ID de otro usuario	403 Forbidden - Do not have permissions for this account	Pass

#2 Obtener los ultimos 5 movimientos de la cuenta (ordenadas por fecha) - GET /accounts/{id}/transactions (account-service) - Suite automatizada: TransactionApiTest

ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC07	Obtener transacciones correctamente	Usuario autenticado con token válido y cuenta con movimientos	Enviar GET /accounts/{id}/transactions con token válido	200 OK + lista de transacciones ordenadas DESC (más reciente primero)	Pass
TC08	Obtener transacciones sin token	API disponible	Enviar GET /accounts/{id}/transactions sin token	401 Unauthorized	Pass
TC09	Obtener transacciones con token inválido	API disponible	Enviar request con token inválido	401 Unauthorized (Invalid Token)	Pass
TC10	Obtener transacciones con ID inexistente	Usuario autenticado	Enviar GET con ID inexistente	404 Not Found - Account not found	Pass
TC11	Obtener transacciones con ID inválido (string)	API disponible	Enviar GET /accounts/abc/transactions	400 Bad Request	Pass

TC12	Usuario accede a transacciones de otra cuenta	Usuario autenticado	GET con ID de otro usuario	403 Forbidden - Do not have permissions	Pass
TC13	Cuenta sin transacciones	Usuario autenticado con cuenta sin movimientos	GET /accounts/{id}/transactions	200 OK + lista vacía	Pass
TC14	Validar límite de 5 transacciones	Usuario con más de 5 transacciones	GET /accounts/{id}/transactions ?limit=5	La respuesta contiene solo 5 elementos	Pass
02 Epica: Mi perfil					
#3 Obtener perfil del usuario – GET /users/{id} (user-service) - Suite automatizada: UserApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC15	Obtener perfil correctamente	Usuario autenticado con token válido	GET /users/10 con Bearer Token y X-User-Id: 10 vía gateway (8083)	200 OK + { id, firstName, lastName, cvu, alias }	Pass
TC16	Obtener perfil sin token	API disponible	Enviar GET /users/10 sin token vía gateway (8083)	401 Unauthorized	Pass
TC17	Obtener perfil con token inválido	API disponible	Enviar request con token inválido vía gateway (8083)	401 Unauthorized (Invalid Token)	Pass
TC18	Acceder al perfil de otro usuario	Usuario autenticado (X-User-Id: 10)	GET /users/9 con X-User-Id: 10 vía gateway (8083)	403 Forbidden – Do not have permissions	Pass
TC19	ID de usuario inexistente	Usuario autenticado. Se ejecuta directamente contra user-service (8084) porque el Gateway intercepta con 403 antes de llegar al microservicio. El 404 es válido a nivel de servicio.	GET /users/9999 con X-User-Id: 9999 vía user-service (8084)	404 Not Found – User not found	Pass
TC20	ID inválido (string)	API disponible	GET /users/abc vía user-service (8083)	400 Bad Request – Invalid user ID format	Pass

TC21	account-service caído	Usuario autenticado, account-service detenido	GET /users/10 con account-service detenido vía gateway (8083)	503 Service Unavailable – Account service unavailable	Pass
#4 Obtener cuenta por userId – GET /accounts/user/{userId} (account-service) - Suite automatizada: AccountUserApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC22	Obtener account correctamente	Usuario autenticado con cuenta existente	GET /accounts/user/10 con X-User-Id: 10	200 OK + { cvu, alias }	Pass
TC23	Acceder a la cuenta de otro usuario	Usuario autenticado (X-User-Id: 10)	GET /accounts/user/9 con X-User-Id: 10	403 Forbidden – Do not have permissions	Pass
TC24	userId sin cuenta asociada	Usuario autenticado sin account creada. Se ejecuta directamente contra account-service (8081) porque el Gateway intercepta con 401 antes de llegar al microservicio. El 404 es válido a nivel de servicio.	GET /accounts/user/9999 con X-User-Id: 9999 via account-service (8081)	404 Not Found – Account not found for userId	Pass
TC25	userId inválido (string)	API disponible	GET /accounts/user/abc	400 Bad Request – Invalid user ID format	Pass
TC26	Error interno en account-service	account-service con falla interna	GET /accounts/user/10, DB no responde	500 Internal Server Error	Pass
03 Epica: Registro de tarjetas					
#5 Crear una tarjeta - POST /cards (card-service) - Suite automatizada: CardApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC27	Crear tarjeta correctamente	Usuario autenticado con token válido	Enviar POST /cards con datos válidos de tarjeta	201 Created + datos de la tarjeta creada	
TC28	Crear tarjeta sin token	API disponible	Enviar POST /cards sin token	401 Unauthorized	Pass
TC29	Crear tarjeta con datos incompletos	API disponible	Enviar POST /cards sin campos obligatorios (ej: número, vencimiento)	400 Bad Request	Pass

TC30	Crear tarjeta con número inválido	API disponible	Enviar POST /cards con número de tarjeta en formato incorrecto	400 Bad Request	Pass
TC31	Error interno al crear tarjeta	Falla en el servidor / base de datos	Enviar POST /cards con datos válidos	500 Internal Server Error	Pass
#6 Asociar tarjeta a una cuenta - POST /cards/accounts/{id}/cards (card-service) - Suite automatizada: CardAssociateApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC32	Asociar tarjeta correctamente	Usuario autenticado, cuenta y tarjeta existentes	Enviar POST /accounts/{id}/cards con token válido e ID de tarjeta	201 Created + relación cuenta-tarjeta	Pass
TC33	Asociar tarjeta sin token	API disponible	Enviar POST /accounts/{id}/cards sin token	401 Unauthorized	Pass
TC34	Asociar tarjeta a cuenta inexistente	Usuario autenticado	Enviar POST con ID de cuenta inexistente	404 Not Found - Account not found	Pass
TC35	Asociar tarjeta ya asociada a otra cuenta	Usuario autenticado, tarjeta previamente asociada	Enviar POST /accounts/{id}/cards con tarjeta ya vinculada	409 Conflict - Card already associated	Pass
TC36	Asociar tarjeta con ID inválido (string)	API disponible	Enviar POST /accounts/abc/cards	400 Bad Request	Pass
TC37	Asociar tarjeta de otro usuario	Usuario autenticado	Enviar POST con tarjeta perteneciente a otro usuario	403 Forbidden - Do not have permissions	Pass
TC38	Error interno al asociar tarjeta	Falla en el servidor / base de datos	Enviar POST con datos válidos	500 Internal Server Error	Pass
#7 Buscar tarjetas asociadas a una cuenta - GET /cards/accounts/{id}/cards (card-service) - Suite automatizada: CardListApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC39	Obtener tarjetas correctamente	Usuario autenticado con token válido y cuenta con tarjetas asociadas	Enviar GET /accounts/{id}/cards con token válido	200 OK + lista de tarjetas asociadas a la cuenta	Pass
TC40	Obtener tarjetas sin token	API disponible	Enviar GET /accounts/{id}/cards sin token	401 Unauthorized	Pass

TC41	Obtener tarjetas con token inválido	API disponible	Enviar request con token inválido	401 Unauthorized (Invalid Token)	Pass
TC42	Cuenta sin tarjetas asociadas	Usuario autenticado con cuenta sin tarjetas	Enviar GET /accounts/{id}/cards con token válido	200 OK + lista vacía	Pass
TC43	Obtener tarjetas con ID inválido (string)	API disponible	Enviar GET /accounts/abc/cards	400 Bad Request	Pass
TC44	Usuario accede a tarjetas de otra cuenta	Usuario autenticado	GET con ID de cuenta de otro usuario	403 Forbidden - Do not have permissions	Pass
TC45	Obtener tarjetas con ID inexistente	Usuario autenticado	Enviar GET con ID de cuenta inexistente	404 Not Found - Account not found	Pass
TC46	Error interno al obtener tarjetas	Falla en el servidor / base de datos	Enviar GET /accounts/{id}/cards con datos válidos	500 Internal Server Error	Pass

#8 Obtener detalle de una tarjeta asociada a una cuenta - GET /accounts/{accountId}/cards/{cardId} (card-service) - Suite automatizada: CardDetailApiTest

ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC47	Obtener detalle de tarjeta correctamente	Usuario autenticado con token válido, cuenta y tarjeta existentes	Enviar GET /accounts/{accountId}/cards/{cardId} con token válido	200 OK + detalle completo de la tarjeta	Pass
TC48	Obtener detalle sin token	API disponible	Enviar GET /accounts/{accountId}/cards/{cardId} sin token	401 Unauthorized	Pass
TC49	Obtener detalle con token inválido	API disponible	Enviar request con token inválido	401 Unauthorized (Invalid Token)	Pass
TC50	ID de cuenta inválido (string)	API disponible	Enviar GET /accounts/abc/cards/{cardId}	400 Bad Request	Pass
TC51	ID de tarjeta inválido (string)	API disponible	Enviar GET /accounts/{accountId}/cards/abc	400 Bad Request	Pass
TC52	Cuenta inexistente	Usuario autenticado	Enviar GET con ID de cuenta inexistente	404 Not Found - Account not found	Pass

TC53	Tarjeta inexistente	Usuario autenticado con cuenta válida	Enviar GET con ID de tarjeta inexistente	404 Not Found - Card not found	Pass
TC54	Tarjeta no asociada a la cuenta	Usuario autenticado con cuenta y tarjeta existentes	Enviar GET con tarjeta que no pertenece a la cuenta indicada	404 Not Found - Card not found	Pass
TC55	Usuario accede a tarjeta de otra cuenta	Usuario autenticado	GET con ID de cuenta de otro usuario	403 Forbidden - Do not have permissions	Pass
TC56	Error interno al obtener detalle	Falla en el servidor / base de datos	Enviar GET con datos válidos	500 Internal Server Error	Pass
#9 Eliminar una tarjeta - DELETE /accounts/{accountId}/cards/{cardId} (card-service) - Suite automatizada: CardDetailApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC57	Eliminar tarjeta correctamente	Usuario autenticado con token válido, cuenta y tarjeta existentes	Enviar DELETE /accounts/{accountId}/cards/{cardId} con token válido	200 OK - Tarjeta eliminada correctamente	Pass
TC58	Eliminar tarjeta sin token	API disponible	Enviar DELETE /accounts/{accountId}/cards/{cardId} sin token	401 Unauthorized	Pass
TC59	Eliminar tarjeta con token inválido	API disponible	Enviar request con token inválido	401 Unauthorized (Invalid Token)	Pass
TC60	ID de cuenta inválido (string)	API disponible	Enviar DELETE /accounts/abc/cards/{cardId}	400 Bad Request	Pass
TC61	ID de tarjeta inválido (string)	API disponible	Enviar DELETE /accounts/{accountId}/cards/abc	400 Bad Request	Pass
TC62	Cuenta inexistente	Usuario autenticado	Enviar DELETE con ID de cuenta inexistente	404 Not Found - Account not found	Pass
TC63	Tarjeta inexistente	Usuario autenticado con cuenta válida	Enviar DELETE con ID de tarjeta inexistente	404 Not Found - Card not found	Pass
TC64	Tarjeta no asociada a la cuenta	Usuario autenticado con cuenta y tarjeta existentes	Enviar DELETE con tarjeta que no pertenece a la cuenta indicada	404 Not Found - Card not found	Pass
TC65	Usuario elimina tarjeta de otra cuenta	Usuario autenticado	DELETE con ID de cuenta de otro usuario	403 Forbidden - Do not have permissions	Pass

TC66	Eliminar tarjeta ya eliminada	Usuario autenticado con cuenta válida	Enviar DELETE con ID de tarjeta previamente eliminada	404 Not Found - Card not found	Pass
TC67	Error interno al eliminar tarjeta	Falla en el servidor / base de datos	Enviar DELETE con datos válidos	500 Internal Server Error	Pass
02 Epica: Mi perfil (parte de la funcionalidad que faltaba)					
#10 Actualizar perfil de usuario – PATCH /users/{id} (user-service) - Suite automatizada: UserPatchApiTest					
ID	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC68	Actualizar perfil correctamente	Usuario autenticado con token válido	Enviar PATCH /users/{id} con token válido y body { firstName, lastName }	200 OK + datos actualizados	Pass
TC69	Actualizar perfil sin token	API disponible	Enviar PATCH /users/{id} sin token	401 Unauthorized	Pass
TC70	Actualizar perfil con token inválido	API disponible	Enviar PATCH /users/{id} con token inválido	401 Unauthorized (Invalid Token)	Pass
TC71	Actualizar perfil de otro usuario	Usuario autenticado (X-User-Id: 10)	Enviar PATCH /users/9 con token del usuario 10	403 Forbidden - Do not have permissions	Pass
TC72	Campos vacíos o faltantes	Usuario autenticado con token válido	Enviar PATCH /users/{id} con body vacío o sin campos obligatorios	400 Bad Request	Pass
#11 Actualizar alias de cuenta – PATCH /accounts/{id} (account-service) - Suite automatizada: AccountPatchApiTest					
	Caso de prueba	Precondición	Pasos	Resultado esperado	Estado
TC73	Actualizar alias correctamente	Usuario autenticado con token válido y cuenta existente	Enviar PATCH /accounts/{id} con token válido y body { alias }	200 OK + nuevo alias	Pass
TC74	Actualizar alias sin token	API disponible	Enviar PATCH /accounts/{id} sin token	401 Unauthorized	Pass
TC75	Actualizar alias con token inválido	API disponible	Enviar PATCH /accounts/{id} con token inválido	401 Unauthorized (Invalid Token)	Pass
TC76	Actualizar cuenta de otro usuario	Usuario autenticado (X-User-Id: 10)	Enviar PATCH /accounts/{id} con ID de cuenta de otro usuario	403 Forbidden - Do not have permissions	Pass

TC77	Alias ya en uso	Usuario autenticado con cuenta existente	Enviar PATCH /accounts/{id} con alias ya registrado en otra cuenta	409 Conflict - Alias already in use	Pass
TC78	Alias vacío o faltante	Usuario autenticado con token válido	Enviar PATCH /accounts/{id} con body vacío o sin campo alias	400 Bad Request	Pass